



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

Vicerrectorado Académico

Coordinación General de Pregrado

Ingeniería en Informática

Lenguajes y Compiladores

## **MECANISMOS QUE INTERACTÚAN EN WASM Y SU IMPORTANCIA EN TIEMPO DE RESPUESTA DE LAS API**

### **Tópico 15**

Grupo: Compiladores CLRR

Eslogan: Los que mas compilan

**Profesor:**

Félix Márquez

**Integrantes:**

Rafael Rodriguez - C.I.: 31882367 - Sec. 1

Adrian Reina - C.I.: 31317970 - Sec. 1

Fernando Centeno - C.I.: 31810484 - Sec. 1

Juan Longart - C.I.: 31882343 - Sec. 1

Ciudad Guayana, Mayo 2026

## **ÍNDICE**

|                                                    |    |
|----------------------------------------------------|----|
| Introducción .....                                 | 3  |
| Marco Teórico: WebAssembly .....                   | 4  |
| Mecanismos Internos de WASM .....                  | 5  |
| WASM y las APIs: Mecanismos de Interacción .....   | 8  |
| Impacto en Tiempos de Respuesta de las API .....   | 10 |
| WASM en Backend: Edge Computing y Serverless ..... | 12 |
| Análisis Comparativo .....                         | 14 |
| Casos de Uso Reales .....                          | 15 |
| Conclusiones .....                                 | 17 |
| Referencias .....                                  | 18 |

## **1. Introducción**

WebAssembly (WASM) ha emergido como una de las tecnologías más transformadoras en el desarrollo de software moderno. Desde su estandarización por el W3C en 2019, WASM ha redefinido los límites de lo que es posible ejecutar tanto en navegadores web como en entornos de servidor, ofreciendo un rendimiento cercano al código nativo manteniendo la portabilidad y seguridad que demandan las aplicaciones contemporáneas (Haas et al., 2017).

En el contexto de las Interfaces de Programación de Aplicaciones (APIs), los tiempos de respuesta constituyen un factor crítico que determina la experiencia del usuario y la eficiencia operativa de los sistemas. La latencia en las APIs puede impactar directamente en la percepción de calidad del servicio, la retención de usuarios y los costos operativos de infraestructura (Google, 2023).

El presente informe investiga los mecanismos internos que interactúan en WebAssembly y analiza su importancia en la optimización de los tiempos de respuesta de las APIs. Se examinan los componentes arquitectónicos fundamentales de WASM, los mecanismos de interoperabilidad con los entornos host, y se presentan benchmarks y casos de uso reales que demuestran su impacto en el rendimiento de las aplicaciones.

La investigación abarca desde los fundamentos teóricos de la máquina virtual basada en pila de WASM, hasta las implementaciones modernas en edge computing y serverless, proporcionando una visión integral de cómo esta tecnología está redefiniendo el panorama del desarrollo de APIs de alto rendimiento.

## **2. Marco Teórico: WebAssembly**

### **2.1 Definición y Origen**

WebAssembly (abreviado WASM) es un estándar abierto que define un formato de instrucción binaria para una máquina virtual basada en pila (stack-based virtual machine). Fue diseñado como un objetivo de compilación portátil para lenguajes de programación de alto nivel como C, C++, Rust, Go y Kotlin, permitiendo que el código se ejecute con rendimiento cercano al nativo en múltiples plataformas (Mozilla Developer Network, 2024).

El proyecto fue iniciado en 2015 por un esfuerzo colaborativo entre Mozilla, Google, Microsoft y Apple, con el objetivo de superar las limitaciones de rendimiento de JavaScript para tareas computacionalmente intensivas. En 2017, los principales navegadores (Chrome, Firefox, Safari y Edge) lanzaron soporte inicial para WASM, y en 2019 fue oficialmente estandarizado como recomendación del W3C (World Wide Web Consortium, 2019).

### **2.2 Objetivos de Diseño**

Los objetivos fundamentales de WebAssembly son cuatro: (1) ser rápido, eficiente y portátil, ejecutándose a velocidades cercanas al código nativo aprovechando capacidades comunes del hardware; (2) ser legible y depurable, definiendo un formato de texto legible para depuración y pruebas; (3) ser seguro, ejecutándose en un entorno aislado (sandbox) sin acceso directo al sistema operativo; y (4) no romper la web, diseñado para coexistir con JavaScript y aprovechar las APIs web existentes (WebAssembly.org, 2024).

### **2.3 El Estándar W3C y su Evolución**

La especificación de WebAssembly ha evolucionado significativamente. La versión 1.0 (2019) definió el formato binario base, el modelo de ejecución y la API JavaScript. La versión 2.0 (2022) introdujo instrucciones SIMD de 128 bits, tipos de referencia y múltiples tablas. En septiembre de 2025, la versión 3.0 formalizó el soporte nativo de recolección de basura (WasmGC), mejorando drásticamente el rendimiento de lenguajes como Java, Kotlin y Dart compilados a WASM (W3C, 2025).

### **3. Mecanismos Internos de WASM**

#### **3.1 El Módulo WASM**

El módulo es la unidad fundamental de despliegue y ejecución en WebAssembly, representado como un archivo binario con extensión `.wasm`. Un módulo es autocontenido e incluye funciones, memoria, tablas y variables globales. Puede importar funciones del entorno host (como JavaScript en un navegador) y exportar sus propias funciones para ser invocadas externamente. Antes de ejecutarse, cada módulo pasa por un proceso de validación estricta que garantiza la integridad de tipos y la seguridad del flujo de control (Mozilla Developer Network, 2024).

#### **3.2 Máquina Virtual Basada en Pila (Stack Machine)**

WebAssembly opera internamente como una máquina de pila. Las instrucciones extraen (pop) operandos de la parte superior de una pila de ejecución, realizan la operación correspondiente y depositan (push) el resultado de vuelta en la pila. Este diseño fue elegido por tres razones fundamentales: es extremadamente compacto (reduce el tamaño del binario), es fácil de validar formalmente (garantiza seguridad de tipos), y se mapea eficientemente a las instrucciones de procesadores reales mediante compilación JIT o AOT (Haas et al., 2017).

Por ejemplo, una operación de suma en WASM se expresa como: primero se colocan dos valores enteros en la pila (`i32.const 5`, `i32.const 3`), luego la instrucción `i32.add` extrae ambos valores, los suma y coloca el resultado (8) en la pila. Este modelo contrasta con las máquinas basadas en registros utilizadas por arquitecturas como x86 o ARM.

#### **3.3 Memoria Lineal**

La memoria lineal es un bloque contiguo, plano y mutable de bytes que el módulo WASM utiliza para almacenar datos. Funciona conceptualmente como un `ArrayBuffer` de JavaScript: el código accede a la memoria mediante offsets (índices) y operaciones de carga/almacenamiento (`load/store`). La memoria puede crecer dinámicamente mediante la instrucción `memory.grow`, pero siempre mantiene su naturaleza contigua (Mozilla Developer Network, 2024).

Este mecanismo es fundamental para el rendimiento porque permite acceso directo y predecible a la memoria sin la sobrecarga de un recolector de basura. Sin embargo, todo acceso está limitado a los bounds definidos del módulo: cualquier intento de acceder fuera de los límites provoca un trap (interrupción inmediata), garantizando la seguridad del sandbox.

### **3.4 Tablas de Funciones**

Las tablas son estructuras que almacenan referencias tipadas, principalmente punteros a funciones. Dado que la memoria lineal contiene bytes sin tipo, almacenar punteros a funciones directamente en ella sería inseguro. Las tablas resuelven este problema proporcionando un mecanismo seguro para llamadas indirectas a funciones (`call_indirect`), esencial para implementar patrones como tablas de métodos virtuales (`vtables`), callbacks y despacho dinámico (WebAssembly.org, 2024).

### **3.5 Compilación: AOT y JIT**

Los motores de ejecución modernos emplean dos estrategias de compilación para WASM. La compilación Ahead-of-Time (AOT) traduce el bytecode WASM a código máquina nativo antes de la ejecución, eliminando la latencia de compilación del camino crítico. La compilación Just-in-Time (JIT) compila el código durante la ejecución, permitiendo optimizaciones basadas en el perfil de uso real. Motores como V8 (Chrome) utilizan un enfoque de dos niveles: Liftoff para compilación rápida inicial y TurboFan para optimización agresiva posterior (Google V8 Team, 2024).

### **3.6 Validación y Seguridad (Sandboxing)**

Todo módulo WASM se ejecuta dentro de un sandbox estricto. No tiene acceso directo al sistema operativo, la red, el sistema de archivos ni la memoria del proceso host. Para interactuar con el mundo exterior, debe utilizar interfaces explícitamente definidas: la API JavaScript en navegadores o WASI (WebAssembly System Interface) en entornos de servidor. Este modelo de seguridad por capacidades (*capability-based security*) es una ventaja fundamental sobre los contenedores tradicionales (Bytecode Alliance, 2024).

## **4. WASM y las APIs: Mecanismos de Interacción**

### **4.1 Interoperabilidad WASM-JavaScript**

En el contexto del navegador, WASM interactúa con las APIs web a través de un puente con JavaScript. Las funciones exportadas por un módulo WASM pueden ser invocadas desde JavaScript como funciones regulares, y viceversa, un módulo puede importar funciones JavaScript. Este mecanismo permite que WASM procese datos computacionalmente intensivos mientras JavaScript maneja la lógica de I/O, DOM y red (Mozilla Developer Network, 2024).

Sin embargo, cada llamada que cruza la frontera WASM-JS incurre en un costo de interoperabilidad típicamente entre 2 y 5 microsegundos. Para aplicaciones con millones de llamadas pequeñas y frecuentes, este overhead puede degradar el rendimiento por debajo de una implementación pura en JavaScript. La estrategia óptima consiste en minimizar las llamadas transfronterizas agrupando (batching) los datos transferidos (Smith, 2024).

### **4.2 WASI: WebAssembly System Interface**

WASI es una interfaz estandarizada que permite a los módulos WASM interactuar con recursos del sistema operativo de manera portable y segura. La versión WASI Preview 2 (también conocida como WASI 0.2), estabilizada en 2024, introdujo el Component Model que permite la composición modular de aplicaciones. WASI 0.3, en desarrollo durante 2025, agrega soporte nativo para I/O asíncrono, esencial para el manejo eficiente de APIs concurrentes (Bytecode Alliance, 2025).

WASI opera bajo un modelo de seguridad por capacidades: un módulo solo puede acceder a los recursos que le son explícitamente concedidos por el host. Esto contrasta con el modelo tradicional de permisos basados en usuario del sistema operativo, proporcionando un aislamiento granular ideal para ejecutar código de terceros en APIs multi-tenant.

### **4.3 El Component Model**

El Component Model es una extensión del estándar WASM que permite definir interfaces tipadas (WIT - WebAssembly Interface Type) entre componentes. Esto posibilita que componentes escritos en diferentes lenguajes (Rust, Go, Python, JavaScript) se comuniquen entre sí sin recurrir a serialización JSON costosa ni FFI manual. Para las APIs, esto significa que los microservicios pueden componerse de manera eficiente con comunicación de baja latencia entre componentes (Bytecode Alliance, 2025).

#### **4.4 SIMD (Single Instruction, Multiple Data)**

Las instrucciones SIMD de 128 bits, soportadas en todos los navegadores principales desde finales de 2024, permiten procesar múltiples datos en paralelo con una sola instrucción. Esto es especialmente relevante para APIs que procesan datos multimedia, realizan operaciones criptográficas o ejecutan inferencia de modelos de machine learning. Los benchmarks demuestran mejoras adicionales de hasta 6x sobre WASM estándar en operaciones vectorizables (WebAssembly Community Group, 2024).



## **5. Impacto en Tiempos de Respuesta de las API**

### **5.1 Benchmarks: WASM vs JavaScript**

Las mediciones de rendimiento consistentes entre 2024 y 2025 demuestran que para tareas computacionalmente intensivas, WASM supera a JavaScript por un factor de 5x a 20x dependiendo de la tarea específica y el nivel de optimización. En procesamiento de imágenes, algoritmos criptográficos y cálculos matemáticos complejos, WASM alcanza velocidades de ejecución dentro del 10-20% del código nativo compilado (Smith, 2024).

No obstante, para operaciones de I/O (llamadas de red, consultas a bases de datos, manipulación del DOM), JavaScript mantiene ventaja o igualdad de rendimiento. La razón es que WASM no tiene acceso directo a estos recursos y debe comunicarse a través del host, añadiendo latencia de interoperabilidad. El consenso de la industria en 2025 favorece un modelo híbrido: JavaScript como capa de orquestación y WASM como motor de rendimiento para cálculos específicos (Google, 2024).

### **5.2 Cold Start y Latencia de Inicialización**

El cold start es el tiempo requerido para compilar e instanciar un módulo WASM antes de la primera ejecución. En navegadores, este tiempo oscila entre 50-150ms para módulos grandes, lo cual puede ser significativo para APIs con requisitos de latencia sub-100ms. Sin embargo, una vez compilado, el código se ejecuta sin penalización adicional. Los runtimes modernos como Wasmtime y Wasmmer han reducido el cold start a menos de 1ms para módulos precompilados (AOT), comparado con los 100ms-5s típicos del arranque de contenedores (Bytecode Alliance, 2024).

### **5.3 Overhead de Frontera (Boundary Crossing)**

Cada transición entre el contexto WASM y el contexto del host genera un overhead medible. En el navegador, una llamada JS a WASM toma aproximadamente 2-5 microsegundos. Si bien esto es insignificante para llamadas individuales, puede acumularse en aplicaciones que realizan millones de transiciones por segundo. La mitigación principal consiste en diseñar interfaces gruesas (coarse-grained): transferir bloques grandes de datos en pocas llamadas en lugar de muchos datos pequeños en llamadas frecuentes (Mozilla Developer Network, 2024).

### **5.4 Tamaño del Binario e Impacto en Carga**

Los módulos WASM compilados tienden a ser más grandes que el JavaScript equivalente

optimizado, lo cual impacta el tiempo de descarga inicial. Un módulo WASM típico puede pesar entre 100KB y varios MB, mientras que el JS equivalente comprimido podría ser significativamente menor. Para APIs web, esto implica considerar técnicas de optimización como tree-shaking, eliminación de símbolos de debug y compresión Brotli para minimizar el impacto en el Time-to-First-Byte (TTFB) de la aplicación.

**Tabla 1. Comparativa WASM vs JavaScript vs Contenedores**

| Característica      | WASM                | JavaScript            | Contenedores        |
|---------------------|---------------------|-----------------------|---------------------|
| Velocidad ejecución | Cercana a nativa    | JIT, variable         | Nativa (OS)         |
| Cold Start          | < 50ms              | Instantáneo           | 100ms - 5s          |
| Tamaño binario      | KB - pocos MB       | KB (texto)            | 50MB - 500MB+       |
| Seguridad           | Sandbox estricto    | Sandbox del motor     | Aislamiento OS      |
| Acceso I/O          | Via WASI/host       | Nativo (Node/Browser) | Completo            |
| Portabilidad        | Universal (WASM VM) | Motor JS requerido    | Imagen por arch.    |
| Multithreading      | SharedArrayBuffer   | Web Workers           | Nativo OS           |
| Mejor para APIs     | Cómputo intensivo   | I/O, CRUD, DOM        | Servicios completos |

## **6. WASM en Backend: Edge Computing y Serverless**

### **6.1 Edge Computing con WASM**

El edge computing representa uno de los campos de mayor crecimiento para WASM. Plataformas como Cloudflare Workers, Fastly Compute y Fermion ejecutan módulos WASM en nodos distribuidos geográficamente cerca de los usuarios finales. El resultado son tiempos de respuesta de API inferiores a 10ms para operaciones que típicamente requerirían 100-300ms al enrutarse a servidores centralizados (Fermion, 2025).

La ventaja de WASM sobre los contenedores en edge es triple: (1) arranque prácticamente instantáneo (microsegundos vs segundos), (2) huella de memoria mínima (KB vs cientos de MB), y (3) aislamiento por sandbox sin necesidad de virtualización pesada. Estas características permiten una densidad de carga de trabajo significativamente mayor, reduciendo costos operativos.

### **6.2 Serverless con WASM**

Los frameworks serverless basados en WASM han madurado considerablemente. Fermion Spin proporciona un framework para construir aplicaciones serverless event-driven que se compilan a WASM. Cada solicitud HTTP puede manejarse con una instancia WASM aislada que se crea y destruye en microsegundos, eliminando el problema de cold start que afecta a las funciones Lambda tradicionales basadas en contenedores (Fermion, 2025).

### **6.3 Runtimes de WASM en Servidor**

Tres runtimes dominan el ecosistema de servidor en 2025. Wasmtime, la implementación de referencia de la Bytecode Alliance, enfatiza la conformidad con estándares y la seguridad. Wasmmer ofrece flexibilidad con múltiples backends de compilación (incluyendo LLVM). WasmEdge se especializa en edge computing y AI inference. Todos soportan WASI Preview 2 y ofrecen rendimiento competitivo para el procesamiento de APIs de alto throughput (Bytecode Alliance, 2025).

### **6.4 SpinKube: WASM en Kubernetes**

SpinKube, un proyecto sandbox de la CNCF, permite ejecutar cargas de trabajo WASM dentro de clusters Kubernetes. Esto posibilita que las organizaciones adopten WASM gradualmente, ejecutando microservicios basados en WASM junto a contenedores tradicionales. Los benchmarks muestran que los pods WASM consumen hasta 10x menos memoria y arrancan hasta 100x más rápido que los pods basados en contenedores

convencionales (CNCF, 2025). *Mecanismos WASM y su importancia en APIs*

## **7. Casos de Uso Reales**

### **7.1 Figma: Motor de Renderizado**

Figma es el caso de estudio más emblemático de WebAssembly. Su motor de renderizado, originalmente escrito en C++, se compila a WASM para ejecutarse en el navegador. Esto permite manejar operaciones de diseño complejas, transformaciones matemáticas y renderizado gráfico a 60 fps sin depender de la interpretación de JavaScript. El resultado es una experiencia equivalente a una aplicación de escritorio nativa, incluyendo colaboración en tiempo real fluida (Figma Engineering, 2023).

### **7.2 Google Earth: Visualización 3D**

Google Earth migró su base de código C++ de escritorio a la web utilizando WASM. La tecnología permite streaming de datos geoespaciales masivos, descompresión en tiempo real y cálculos 3D complejos en hilos de fondo (multithreading). WASM hizo posible ofrecer navegación 3D fluida directamente en el navegador con una reducción significativa del uso de memoria respecto a la versión basada en Native Client (Google, 2024).

### **7.3 Shopify Functions: Lógica de Backend**

Shopify utiliza WASM de manera innovadora en su infraestructura de servidor mediante Shopify Functions. Los desarrolladores escriben lógica personalizada (descuentos, reglas de envío, validaciones de checkout) en Rust o TypeScript que se compila a WASM. El código se ejecuta en un sandbox aislado con latencia mínima en procesos críticos de pago, superando las limitaciones de rendimiento de las llamadas a APIs externas tradicionales (Shopify Engineering, 2024).

### **7.4 Fermion + Akamai: APIs en el Edge**

La integración entre Fermion y Akamai en 2025 permite desplegar funciones serverless WASM en la red global de edge de Akamai. Esto posibilita APIs con tiempos de respuesta sub-10ms y una densidad de carga de trabajo significativamente mayor que las arquitecturas basadas en contenedores, reduciendo tanto la latencia percibida por el usuario como los costos de infraestructura (Fermion, 2025).

## **8. Conclusiones**

WebAssembly representa un avance fundamental en la forma en que se construyen y despliegan las APIs modernas. Los mecanismos internos de WASM, desde su máquina virtual basada en pila hasta su sistema de memoria lineal y su modelo de seguridad por sandbox, proporcionan una base arquitectónica que permite ejecución de código con rendimiento cercano al nativo manteniendo la portabilidad y seguridad requeridas por las aplicaciones distribuidas.

En cuanto al impacto en tiempos de respuesta, la evidencia demuestra que WASM ofrece mejoras significativas (5x-20x) para el procesamiento computacionalmente intensivo dentro del pipeline de una API. Sin embargo, no es una solución universal: las operaciones de I/O y las llamadas frecuentes a través de la frontera WASM-host pueden introducir overhead que contrarresta los beneficios de rendimiento. El modelo óptimo es híbrido, donde WASM acelera los cálculos críticos mientras el entorno host gestiona la red y el I/O.

Las tecnologías complementarias como WASI, el Component Model y los runtimes serverless (Fermion Spin, Wasmtime) están extendiendo el alcance de WASM más allá del navegador hacia el edge computing y la nube. Los cold starts prácticamente inexistentes y la huella de memoria mínima posicionan a WASM como una alternativa viable y superior a los contenedores para funciones serverless y microservicios de baja latencia.

Los casos de uso de empresas como Figma, Google, Shopify y Fermion demuestran que WASM ya no es una promesa futura sino una realidad productiva. A medida que la especificación evoluciona con WasmGC, SIMD extendido y soporte asíncrono nativo en WASI 0.3, el impacto de WebAssembly en el rendimiento de las APIs continuará creciendo, consolidándose como un componente esencial en la arquitectura de software de alto rendimiento.

## Referencias

- Bytecode Alliance. (2024). Wasmtime: A fast and secure runtime for WebAssembly. <https://wasmtime.dev/>
- Bytecode Alliance. (2025). WASI: The WebAssembly System Interface. <https://wasi.dev/>
- CNCF. (2025). SpinKube: Running WebAssembly workloads in Kubernetes. <https://www.spinkube.dev/>
- Fermyon. (2025). Spin: The developer tool for building WebAssembly microservices. <https://www.fermyon.com/spin>
- Figma Engineering. (2023). Building a high-performance design tool with WebAssembly. <https://www.figma.com/blog/webassembly-cut-figmas-load-time-by-3x/>
- Google. (2024). Google Earth and WebAssembly: Bringing 3D to the browser. <https://web.dev/case-studies/earth-webassembly>
- Google V8 Team. (2024). V8 WebAssembly compilation pipeline. <https://v8.dev/blog/wasm-compilation-pipeline>
- Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., & Bastien, J. (2017). Bringing the web up to speed with WebAssembly. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, 185-200.
- Mozilla Developer Network. (2024). WebAssembly concepts. <https://developer.mozilla.org/en-US/docs/WebAssembly/Concepts>
- Shopify Engineering. (2024). Shopify Functions: Extending commerce with WebAssembly. <https://shopify.engineering/shopify-functions-wasm>
- Smith, L. (2024). WebAssembly vs JavaScript: Performance benchmarks in 2024. Plain English. <https://plainenglish.io/blog/wasm-vs-javascript-performance>
- W3C. (2025). WebAssembly Specification 3.0. <https://www.w3.org/TR/wasm-core-3/>
- WebAssembly Community Group. (2024). SIMD proposal for WebAssembly. <https://github.com/WebAssembly/simd>
- WebAssembly.org. (2024). WebAssembly overview. <https://webassembly.org/>
- World Wide Web Consortium. (2019). WebAssembly Core Specification, W3C Recommendation. <https://www.w3.org/TR/wasm-core-1/>